MLVU information sheet

Please include this page in your report either at the start or at the end, before the appendix. Do not change the formatting.

Group number

156

Authors

name	student number
Conrad Clough	2699315
Samuel Power	2800107
ANDI FILL THIS OUT :)	THIS TOOOOO

Software used *Describe briefly which software and libraries you relied on. If you built a particular algorithm from scratch, make sure to state it explicitly here.*

We used Jupyter, numpy, pandas, matplotlib, and sklearn. Jupyter was used for ease of modifying our program without needing to redo data processing; numpy, pandas, and matplotlib were used for interacting with data and displaying results; sklearn was used for their model implementations.

Use of AI tools *If you used tools like ChatGPT or Github Copilot in any phase of the report, please detail here which tools you used, what parts of the work you used them for and how. Note that if you use an AI tool and don't report it, it's considered fraud, so be detailed. Note also that having an AI tool write for you is not allowed. If no AI tools were used, please state that explicitly.*

- Conrad: ChatGPT was used to generate the code that created our graphs, specifically the styling.

Link to code (optional) *If you would like to share your code, you can link to your repository here.*

Group disagreements (optional) *If there are any disagreements in the group, you may* ***not*** *remove a student from the author list without their consent. You should bring disagreements to our attention early. As a last resort, you can describe any grievances here.*

How can machine learning-based DDoS detection models be designed to minimise false positive rates while maintaining high detection accuracy on both low- and high-powered hardware?

Conrad Clough
VUNetId: cch250
Student Nr: 2699315
c.j.clough@student.vu.nl

Samuel Power
VUNetId: spo237
Student Nr: 2800107
s.j.power@student.vu.nl

mr. andy
VUNetId: x
Student Nr: x
lovelyman@student.vu.nl

I. Abstract

https://users.ece.cmu.edu/~koopman/essays/abstract.html

Checklist: Parts of an Abstract

Despite the fact that an abstract is quite brief, it must do almost as much work as the multi-page paper that follows it. In a computer architecture paper, this means that it should in most cases include the following sections. Each section is typically a single sentence, although there is room for creativity. In particular, the parts may be merged or spread among a set of sentences. Use the following as a checklist for your next abstract:

Motivation: Why do we care about the problem and the results? If the problem isn't obviously "interesting" it might be better to put motivation first; but if your work is incremental progress on a problem that is widely recognized as important, then it is probably better to put the problem statement first to indicate which piece of the larger problem you are breaking off to work on. This section should include the importance of your work, the difficulty of the area, and the impact it might have if successful. Problem statement: What problem are you trying to solve? What is the scope of your work (a generalized approach, or for a specific situation)? Be careful not to use too much jargon. In some cases it is appropriate to put the problem statement before the motivation, but usually this only works if most readers already understand why the problem is important. Approach: How did you go about solving or making progress on the problem? Did you use simulation, analytic models, prototype construction, or analysis of field data for an actual product? What was the extent of your work (did you look at one application program or a hundred programs in twenty different programming languages?) What important variables did you control, ignore, or measure? Results: What's the answer? Specifically, most good computer architecture papers conclude that something is so many percent faster, cheaper, smaller, or otherwise better than something else. Put the result there, in numbers. Avoid vague, hand-waving results such as "very", "small", or "significant." If you must be vague, you are only given license to do so when you can talk about orders-of-magnitude improvement. There is a tension here in that you should not provide numbers that can be easily misinterpreted, but on the other hand you don't have room for all the caveats. Conclusions: What are the implications of your answer? Is it going to change the world (unlikely), be a significant "win", be a nice hack, or simply serve as a road sign indicating that this path is a waste of time (all of the previous results are useful). Are your results general, potentially generalizable, or specific to a particular case?

Other Considerations

An abstract must be a fully self-contained, capsule description of the paper. It can't assume (or attempt to provoke) the reader into flipping through looking for an explanation of what is meant by some vague statement. It must make sense all by itself. Some points to consider include:

Meet the word count limitation. If your abstract runs too long, either it will be rejected or someone will take a chainsaw to it to get it down to size. Your purposes will be better served by doing the difficult task of cutting yourself, rather than leaving it to someone else who might be more interested in meeting size restrictions than in representing your efforts in the best possible manner. An abstract word limit of 150 to 200 words is common. Any major restrictions or limitations on the results should be stated, if only by using "weasel-words" such as "might", "could", "may", and "seem". Think of a half-dozen search phrases and keywords that people looking for your work might use. Be sure that those exact phrases appear in your abstract, so that they will turn up at the top of a search result listing. Usually the context of a paper is set by the publication it appears in (for example, IEEE Computer magazine's articles are generally about computer technology). But, if your paper appears in a somewhat untraditional venue, be sure to include in the problem statement the domain or topic area that it is really applicable to. Some publications request "keywords". These have two purposes. They are used to facilitate keyword index searches, which are greatly reduced in importance now that on-line abstract text searching is commonly used. However, they are also used to assign papers to review committees or editors, which can be extremely important to your fate. So make sure that the keywords you pick make assigning your paper to a review category obvious (for example, if there is a list of conference topics, use your chosen topic area as one of the keyword tuples).

Distribution Denial of Service (DDoS) attacks continue to disrupt network services, causing downtime and economic damage. Existing techniques for mitigating this often struggle to whatever they struggle with, find a paper and cite it. This paper investigates the design of DDoS detection models using machine learning. We evaluate three classifiers, k-Nearest Neighbours (kNN), Logistic Regression (LR), and Random Forests, by analysing their accuracy, false positive rates, and resource usage under controlled conditions. Our results indicate that whatever we find. One sentence about each, what they do well and what they do bad. These findings suggest that model selection is dependent on hardware specifications and the relative importance of minimising false positives, providing a practical guide for using machine learning-based DDoS detection in diverse contexts.

Keywords: DDoS detection, machine learning, false positive rate, k-Nearest Neighbours, Random Forest, Logistic Regression, computational efficiency

II. Introduction

The proliferation of online services has made networks increasingly vulnerable to Distributed Denial of Service (DDoS) attacks, which can overwhelm servers and disrupt them, causing significant operational downtime and economic damage. Detecting these attacks efficiently is challenging because high accuracy must be balanced with low false positive rates; denying service to legitimate users is highly undesirable, as is allowing too much malicious traffic through. Machine learning offers a promising alternative to traditional, rule-based approaches; patterns in network traffic can be used to identify evolving malicious activity in real time. This

document investigates the design of machine learning-based DDoS detection models with an emphasis on maintaining high detection accuracy while reducing false positives, and evaluates their resource usage across a range of hardware to ensure usability for a variety of situations. Specifically, k-Nearest Neighbours (kNN), Logistic Regression (LR), and Random Forest classifiers are compared in terms of both accuracy, false positive rate, and performance.

Blah blah blah. We can quote code `like this`. We can also reference sections like this Section II or even define labels on things like figures and images and reference them later.

Wanna reference something in the bibliography? Just do it with another label [1]

The report is structured as follows:

- In Section III we consider existing methods of DDoS detection.
- In Section VIII we discuss the dataset we used and how it was prepared for use in training.
- In Section IV we explain our classifiers and how we ran our experiments.
- In Section V we show our results.
- In Section VI we interpret our results and address what they mean.
- In Section VII we summarise our findings.

III. Related Work

The algorithms in this project were trained on data obtained from kaggle (MAYBE NEED TO LINK HERE??). The data was already mostly equally balanced between benign traffic and DDoS traffic. The data was also shuffled in the event that some unknown order had been preserved upon creation. Notably though the data is quite large and does not fit into the memory of the hardware of the researchers. For this reason, the project employs a chunking approach, so that the data can be fully trained upon. The data is read into the models in chunks. Because of this, the entire dataset is not loaded in at once as not all data points are used. Balancing is done to ensure that training is not hindered by biases. This could for example happen when a model favours the majority class, when trained on imbalanced data. This ensures accurate, fair, and reliable predictions across all classes. In this case, 100 000 rows for DDoS and 100 000 rows for benign traffic were used. The features were also modified. The SimpleImputer function was used to replace cells with no value safely. Some features were also dropped; namely the identifying columns and other non-revelvant determined data were all dropped. The dropped columns were as follows:

Column
`Label`
`Unnamed: 0`
`Flow ID`
`Src IP`
`Dst IP`
`Timestamp`
`Src Port`
`Dst Port`

The features of KNN and LR were also scaled because they depend on distances and coefficient magnitudes. This was not necessary for RF because tree splits are threshold-based and largely scale-invariant. The scaling is important because it ensures that features with larger numerical ranges do not disproportionately influence the model's behavior.

The protocol was additionally one-hot encoded since it is categorical. The data was also cleaned, where inf values were relaced with NaN. For robustness, outliers were identified through use of the median instead of the mean. In terms of training the models, a 60/20/20 split was employed where 60% of the data was used for training, 20% for validation and 20% for testing.

The models that will be evaluated are a k-Nearest Neighbours model (kNN), a random forest model (RF), and a logistical regression (LR) model. Each of these models has tuned hyperparameters. A variety of possible hyper-parameters are tested, and the best one is chosen by the argmax function on the validation set's accuracy. To make sure that the best hyper-parameters aren't simply overfitting the models will also be benchmarked to determine the extent of over or underfitting.

The correctness of the models will be evaluated through the accuracy of the model upon the test data. This mainly takes into account the extent to which false positives were minimised while true positives were maximised. The project will make use of confusion matrices for this. Since these are binary classification models, the receiver operating characteristic (ROC) curve as well as the area under the curve (AUC) will be used to jointly evaluate the extent to which these models are accurate. While ROC is commonly used for datasets that are imbalanced, the project employs upon the already balanced dataset as a metric to be included for the overfitting analyses, which will determine the strengh of the models with respect to the aforementioned research question.

The research question scope includes the use of models on low and high powered hardware. Correctness remains the same as previously defined, with the constraint that the models must run in [XXXXXXXX][TIMEFRAME] to qualify as having successfully run on the specified hardware. To evaluate this, the high powered hardware will encompass a modern computer who's computation power is at least X Y Z, and a low-powered computer like a raspberry-pi who's computation power is maximally X Y Z. In place of a physical raspberry-pi, a raspberry-pi emulator will be used. **WILL BE MEASURUED IN FLOPS**

To strengthen the future findings the report will also make use of additional metrics.

1) Random Forest will make use of the feature importance that is a property of tree-based models. This will assist in the verification that the model's results on testing data are not correlated to a data anomaly that would not allow the model to continue to maintain it's accuracy scores in real world deployment.
2) The logistical regression model will make use of threshold tuning to allow the authors to trade recall

for precision in a controlled way, which could help in reducing false positives. This is important as while both high recall and high precision are important, it is also very important that no real email lands in spam. Perhaps moreso than a spam email landing in the normal inbox.

Add still: how is correctness evaluated? MORE DETAILED, like precision, recall, ROC/AUC etc. feature scaling in KNN and LR (not present in RF), overfitting analysis details, feature importance for RF and why we do that, threshold tuning in LR only, use of SimpleImputer for cleaning, complete hardware XXXX placeholders

– cite 10 papers

[W I L L | F I N I S H | A N D | F O R M A T | S O O N -SAMUEL]

– new kaggle dataset run on thing – Hypotheses - time complexities, hardware

IV. Methodology

V. Results

VI. Discussion

VII. Conclusion

- Bullet points are super easy
- Like this!

1) Numbered lists too
2) Wow
   a) Even with indentation
   - and mixing!

Math, with equation labels:

$$T_{\text{coarse}} = ctf(n) \tag{1}$$

. Or even inline $\frac{n}{2}$ wowie

$$T_{\text{fine}} = \max\left(\left\lceil \frac{c}{2n} \right\rceil \cdot tf(n), tf(n)\right) \tag{2}$$

.

Figures! They CANNOT be in a super directory, like `../plots`, so we have to use `./images/`

In Fig. 1 we can very clearly see that our experimental data matches the time complexity of access/mutation for our structures.

Tables too:

Generally you can surround things in figures or blocks and such, it's all super functional, the typst website will guide you

Typst website also have all the symbol names, and some symbols have subsets like $\subset$ has $\subseteq$

knn does worse on imbalanced because the larger group sizes 'pull' knn into them

find a paper about this

discuss how you can change this, with k tuning or with weights on the classes that represent the imbalance

reference here asdasd [2], [3], [4]

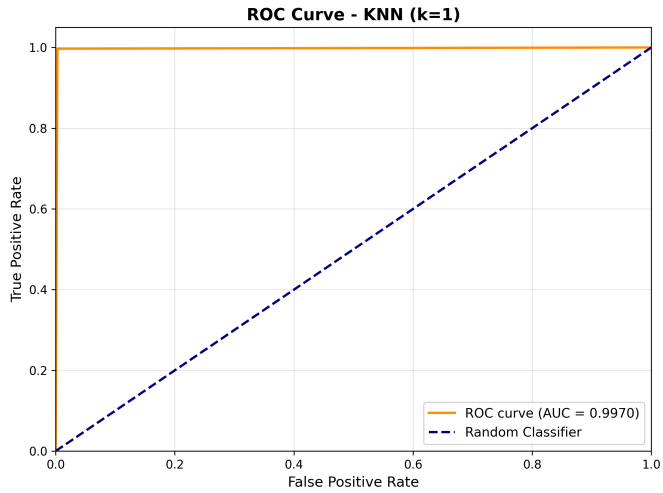

Fig. 1. It's good practice to put figures at the top of the page and just cite them via label

TABLE I
Amdahl's Law metrics at 65536 job count

Structure	p	Max Speedup	Eff. (16 threads)
Coarse-grained List	2.3%	1.02×	6.4%
Coarse-grained BST	92.7%	13.64×	45.4%
Fine-grained List	48.5%	1.94×	22.9%
Fine-grained BST	76.2%	4.20×	19.3%

TABLE II
Resource usage for model training and inference

Model	RAM Δ (MB)	CPU Time (s)	Wall Time (s)
kNN (training)	2034.49	10121.29	1163.05
kNN (inference)	60.95	1402.22	141.90
kNN (inference, Raspberry Pi)	290.60	5344.74	1408.22
Random Forest (training)	2358.29	1619.48	116.39
Random Forest (inference)	6.48	17.46	7.95
Random Forest (inference, Raspberry Pi)	1464.93	51.85	65.14
Logistic Regression (training)	233.83	69.41	42.31
Logistic Regression (inference)	36.73	9.42	7.42
Logistic Regression (inference, Raspberry Pi)	1632.62	27.53	30.78

VIII. Dataset and Preprocessing

Our models were trained on a dataset consisting of network traffic, balanced between DDoS traffic and benign traffic. The data was extracted from public IDS datasets, produced in different years and with different techniques of generating traffic. There are 12 794 627 datapoints and 84 features. We then tested our models' inference abilities on an identically

formed dataset that was unbalanced, with 20% DDoS traffic and 80% benign.

In order to manage resource and time constraints, we limited our training and inference to 200 000 labels. This data was then put into a `pandas` dataframe for efficiency and ease of management. We then dropped particular features that were either irrelevant or non-numeric - both kNN and logistic regression require numeric features. An alternative way of tackling our non-numeric features

Preprocessing can be viewed in detail at [1]. The algorithms in this project were trained on data obtained from kaggle (MAYBE NEED TO LINK HERE??). The data was already mostly equally balanced between benign traffic and DDoS traffic. The data was also shuffled in the event that some unknown order had been preserved upon creation. Notably though the data is quite large and does not fit into the memory of the hardware of the researchers. For this reason, the project employs a chunking approach, so that the data can be fully trained upon. The data is read into the models in chunks. Because of this, the entire dataset is not loaded in at once as not all data points are used. Balancing is done to ensure that training is not hindered by biases. This could for example happen when a model favours the majority class, when trained on imbalanced data. This ensures accurate, fair, and reliable predictions across all classes. In this case, 100 000 rows for DDoS and 100 000 rows for benign traffic were used. The features were also modified. The SimpleImputer function was used to replace cells with no value safely. Some features were also dropped; namely the identifying columns and other non-revelvant determined data were all dropped. The dropped columns were as follows:

Column
`Label`
`Unnamed: 0`
`Flow ID`
`Src IP`
`Dst IP`
`Timestamp`
`Src Port`
`Dst Port`

The features of KNN and LR were also scaled because they depend on distances and coefficient magnitudes. This was not necessary for RF because tree splits are threshold-based and largely scale-invariant. The scaling is important because it ensures that features with larger numerical ranges do not disproportionately influence the model's behavior.

The protocol was additionally one-hot encoded since it is categorical. The data was also cleaned, where inf values were relaced with NaN. For robustness, outliers were identified through use of the median instead of the mean. In terms of training the models, a 60/20/20 split was employed where 60% of the data was used for training, 20% for validation and 20% for testing.

The models that will be evaluated are a k-Nearest Neighbours model (kNN), a random forest model (RF), and a logistical regression (LR) model. Each of these models has tuned hyperparameters. A variety of possible hyper-parameters are tested, and the best one is chosen by the argmax function on the validation set's accuracy. To make sure that the best hyper-parameters aren't simply overfitting the models will also be benchmarked to determine the extent of over or underfitting.

The correctness of the models will be evaluated through the accuracy of the model upon the test data. This mainly takes into account the extent to which false positives were minimised while true positives were maximised. The project will make use of confusion matrices for this. Since these are binary classification models, the receiver operating characteristic (ROC) curve as well as the area under the curve (AUC) will be used to jointly evaluate the extent to which these models are accurate. While ROC is commonly used for datasets that are imbalanced, the project employs upon the already balanced dataset as a metric to be included for the overfitting analyses, which will determine the strengh of the models with respect to the aforementioned research question.

The research question scope includes the use of models on low and high powered hardware. Correctness remains the same as previously defined, with the constraint that the models must run in [XXXXXXXX][TIMEFRAME] to qualify as having successfully run on the specified hardware. To evaluate this, the high powered hardware will encompass a modern computer who's computation power is at least X Y Z, and a low-powered computer like a raspberry-pi who's computation power is maximally X Y Z. In place of a physical raspberry-pi, a raspberry-pi emulator will be used. **WILL BE MEASURUED IN FLOPS**

To strengthen the future findings the report will also make use of additional metrics.

1) Random Forest will make use of the feature importance that is a property of tree-based models. This will assist in the verification that the model's results on testing data are not correlated to a data anomaly that would not allow the model to continue to maintain it's accuracy scores in real world deployment.
2) The logistical regression model will make use of threshold tuning to allow the authors to trade recall for precision in a controlled way, which could help in reducing false positives. This is important as while both high recall and high precision are important, it is also very important that no real email lands in spam. Perhaps moreso than a spam email landing in the normal inbox.

Add still: how is correctness evaluated? MORE DETAILED, like precision, recall, ROC/AUC etc. feature scaling in KNN and LR (not present in RF), overfitting analysis details, feature importance for RF and why we do that, threshold tuning in LR only, use of SimpleImputer for cleaning, complete hardware XXXX placeholders

– cite 10 papers

[W I L L | F I N I S H | A N D | F O R M A T | S O O N -SAMUEL]

– new kaggle dataset run on thing – Hypotheses - time complexities, hardware

References

[1] A. S. C. Clough S. Power, "Project Codebase on Github." [Online]. Available: https://github.com/Flootersmm/ml_project

[2] S. K. S. P. Y. e. a. Rajendran R., "Detection of DoS attacks in cloud networks using intelligent rule based classification system.," *Cluster Computing*, vol. 22, no. ?_number, pp. 423–434, Feb. 2019, doi: 10.1007/s10586-018-2181-4.

[3] S. Dong and M. Sarem, "DDoS Attack Detection Method Based on Improved KNN With the Degree of DDoS Attack in Software-Defined Networks," *IEEE Access*, vol. 8, no. , pp. 5039–5048, 2020, doi: 10.1109/ACCESS.2019.2963077.

[4] A. I. Gide and A. A. M. zu, "A Real-Time Instrusion Detection System for DoS/DDoS Attack Classification in IoT Networks Using KNN-Neural Network Hybrid Technique," *Babylonian Journal of Internet of Things*, vol. 2024, no. , pp. 60–69, 2024, doi: 10.58496/BJIoT/2024/008.